WIZM

A Post-Quantum Payment Chain with Transparent and Unlinkable Stealth Transfers

WIZM Protocol Engineering

29 October 2026

Abstract

WIZM is a five-validator proof-of-authority payment chain written in Rust. It uses Falcon-512 for signatures, ML-KEM for encrypted mempool and transport key establishment, BLAKE3 for hashing, and AES-256-GCM for encrypted mempool payloads. The ledger supports two transaction families: transparent transfers and stealth transfers.

Stealth transfers hide the sender-to-recipient link with a one-time recipient address. The sender and amount are public consensus fields. This paper describes the current transparent and stealth design only.

A block is sealed on a ten-second slot and carries at most 1,200 transactions and 8 MiB. The resulting ceiling is 120 sealed transactions per second. Fees are flat at 10,000 base units, with a per-wallet burst multiplier above 20 transactions in one block.

Contents

1 Design scope **2**

2 Cryptographic primitives **2**
2.1 Signatures . . . 2
2.2 Encrypted mempool and transport . . . 2
2.3 Hashing . . . 2

3 Transactions **2**
3.1 Transparent transfer . . . 2
3.2 Stealth transfer . . . 3

4 Consensus and block limits **3**

5 Ordering, mempool, and finality **3**

6 Tokenomics **4**

7 Wallet and key handling **4**

8 Testing and operations **4**

9 Security limitations **4**

10 Conclusion **5**

1 Design scope

WIZM is a payments ledger. Its security goals are:

1. authenticate every value transfer with a post-quantum signature;
2. keep the recipient of a stealth payment unlinkable to its published meta-address;
3. prevent replay, double spending, and nonce reuse;
4. make block capacity and fee behavior explicit and deterministic;
5. keep the encrypted mempool opaque until block ordering.

The non-goals are equally explicit: hiding the sender or amount in stealth payments, and unlinkability of the transparent tier.

2 Cryptographic primitives

2.1 Signatures

Transactions, validator votes, block seals, and transport authentication use **Falcon-512**. A public key is 897 bytes; a detached signature is variable-length in the range 512–752 bytes.

Addresses are the first 20 bytes of

$$\text{addr}(pk) = \text{BLAKE3}(pk)[0..20].$$

2.2 Encrypted mempool and transport

Mempool entries are encrypted under the committee's ML-KEM-512 key and AES-256-GCM. Inter-node transport uses an ML-KEM-768 key exchange and authenticated encryption. The encrypted mempool hides transaction contents from observers until ordering; it does not hide balances or transfer amounts.

2.3 Hashing

BLAKE3 is used for transaction hashes, block hashes, address derivation, state accumulation, and scheduling randomness. Domain separators are used where the protocol signs or hashes structured messages.

3 Transactions

All integer fields are little-endian. Wire sizes include the Falcon public key and signature; Falcon signatures are variable-length.

3.1 Transparent transfer

`[from:20][to:20][amount:8][nonce:8][Falcon pk:897][signature:512..752]`

- Wire size: 1,465–1,705 bytes.
- Public sender, receiver, and amount.
- Signature covers `from || to || amount || nonce`.
- Execution enforces the sequential account nonce and sufficient balance.

3.2 Stealth transfer

```
[sender:20][stealth_addr:20][ephemeral:32][view_tag:1]
[amount:8][key_image:32][Falcon pk:897][signature:512..752]
```

- Wire size: 1,522–1,762 bytes.
- Sender and amount are public.
- `stealth_addr` is a one-time recipient address derived from the recipient's spend key and an ephemeral shared value.
- The recipient scans with the view key and the key image prevents a second spend.
- Signature covers the 113-byte body.
- **The amount is a public 8-byte field.** There is no amount encryption in the current wire format.

4 Consensus and block limits

WIZM runs proof-of-authority among five validators. The steady-state slot is 10 seconds. Genesis and slow-start slots use longer durations to allow the mesh to form.

Parameter	Value
Validators	5 seats
Steady slot	10 s
Max transactions per block	1,200
Max block body	8 MiB
Max sealed throughput	120 TPS
Finality	notary majority
Fork resolution	earlier seal; deterministic unwind

A proposal drains at most 1,200 transactions and 8 MiB. The count cap normally binds before the byte cap for transparent and stealth transactions. Block validation and execution enforce the same limits.

5 Ordering, mempool, and finality

The encrypted mempool admits ciphertext entries, maintains per-sender nonce floors, parks gap-held lanes, and drains in a deterministic order. A producer seals a block; validators sign notarization votes for the block hash and height. A majority of active seats advances finality. The finality model is crash-fault, not Byzantine-fault-tolerant state machine replication.

Nodes synchronize blocks through direct unicast and gossip, with bounded pipelines and deterministic fork recovery. Mesh-best state is updated only from locally validated blocks, not from unvalidated peer height claims.

6 Tokenomics

Parameter	Value
Symbol	WIZM
Decimals	8
Supply	approximately 83,120,300 WIZM
Emission	10 years, five 2-year periods
Base fee	10,000 base units (0.0001 WIZM)
Ingress minimum	1,000,000 base units (0.01 WIZM)
Burst-free transactions	20 per wallet per block
Burst multiplier	10x per additional transaction
Current block reward	0.8812645 WIZM per validator

The base fee is charged once per value-bearing transaction. During the emission period fees are burned; after emission they are distributed to validators. The burst multiplier is per wallet and per block. A wallet that submits more than 20 fee-paying transactions in one block pays 10x for the 21st, 100x for the 22nd, 1,000x for the 23rd, and so on until the fee becomes unpayable. Load generators must keep every wallet below this threshold.

7 Wallet and key handling

The native wallet stores keys under `~/.wizm/wallets` with directory mode `0700` and file mode `0600`. The Tauri backend exposes explicit wallet list/save/delete commands. The browser wallet uses session-only storage; legacy plaintext wallet records are migrated to native storage or removed. Wallet backups use PBKDF2-SHA256 and AES-256-GCM.

Stealth amounts are public. The user interface displays the unlinkability scope explicitly.

8 Testing and operations

The Rust test suite covers cryptography, transaction layouts, admission, execution, tokenomics, mempool behavior, consensus state, storage, and API surfaces. The current suite contains more than 400 tests. Load tests use transparent and stealth transactions only. A ten-day mixed soak is configured with 50–150 TPS offered phases and an approximately 120 TPS offered average; the chain's own proposal cap remains 120 TPS, so high phases intentionally exercise mempool backlog and low phases drain it.

9 Security limitations

WIZM does not hide balances or transfer amounts. In particular:

- transparent transfers expose sender, receiver, and amount;
- stealth transfers expose sender and amount; only the recipient link is hidden;
- validators and mempool committee members can process plaintext amounts after decryption/ordering;
- independent security review and operational key management are still required for production use.

10 Conclusion

WIZM is a post-quantum, five-validator payment chain with explicit capacity, deterministic fees, an encrypted mempool, transparent transfers, and unlinkable stealth recipients. It deliberately does not claim that balances or amounts are hidden. Its security and performance claims are bounded by the wire formats, block limits, and cryptographic primitives specified here.